세계 최고 수준의
데이터 파이프라인 시스템 구축 제안서

Python + PostgreSQL 기반 / API · OCR · DB-to-DB · 크롤링 · 클라우드소싱 · Visual ETL · 관제

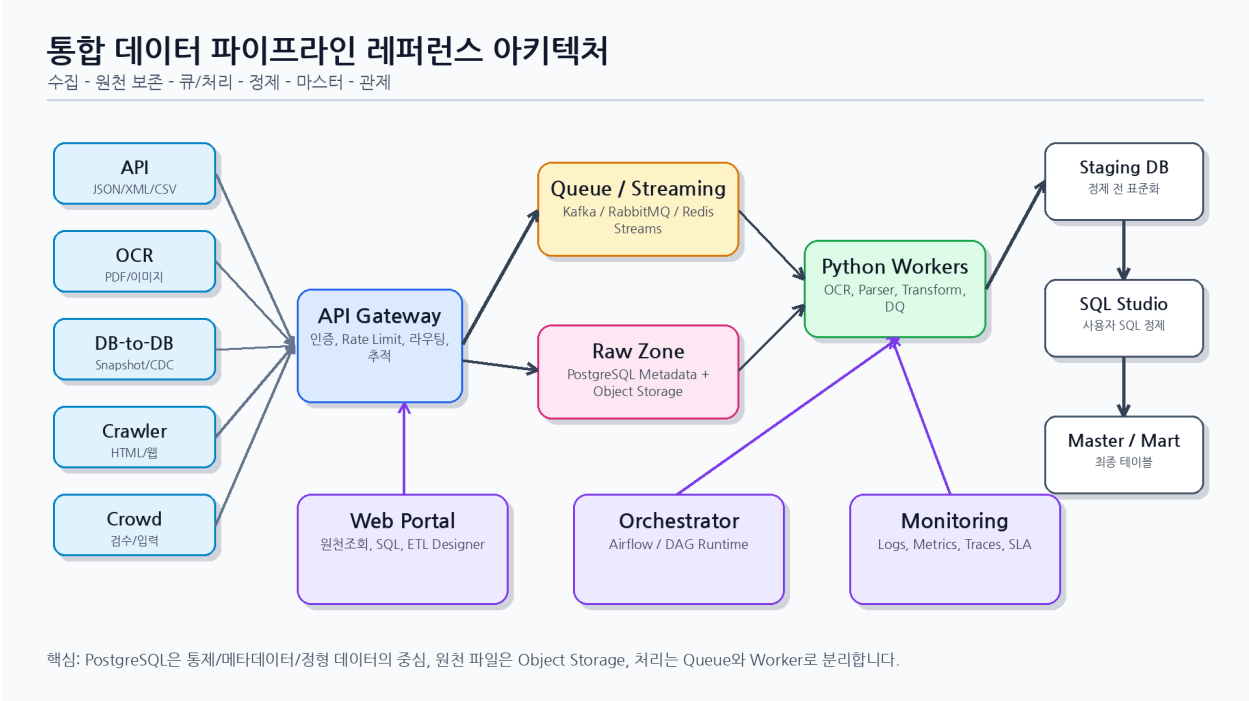

그림 1. 통합 데이터 파이프라인 레퍼런스 아키텍처

작성일: 2026-04-24

1. Executive Summary

이 설계서는 Python과 PostgreSQL을 중심으로 통합 데이터 수집·정제·관제 플랫폼을 구축하기 위한 제안서입니다. 핵심은 PostgreSQL을 모든 원천 파일을 담는 단일 저장소로 쓰는 것이 아니라, 메타데이터, 정형화 데이터, 마스터 데이터, 작업 이력, 감사 로그의 중심 DB로 사용하는 것입니다. PDF, 이미지, HTML, 대형 CSV 같은 원천 파일은 Object Storage에 저장하고, PostgreSQL에는 URI, 해시, 상태, 품질 결과, lineage를 남깁니다.

사용자는 웹 화면에서 데이터 소스와 수집 작업을 관리하고, 입수된 원천 데이터를 확인하며, SQL로 정제하고, SAS Enterprise Guide처럼 사각형 노드를 연결해 ETL 흐름을 설계할 수 있습니다. 배치가 시작되면 각 노드는 대기, 실행중, 완료, 오류 상태로 실시간 갱신됩니다.

핵심 제안: 데이터를 많이 넣는 시스템이 아니라, 어떤 데이터가 언제 어디서 들어와 어떤 규칙으로 정제되어 어떤 마스터 테이블에 반영되었는지 끝까지 추적 가능한 플랫폼을 구축합니다.

2. 목표 시스템 개요

2.1 시스템명

Unified Data Pipeline Platform

2.2 핵심 목표

- API, OCR, DB-to-DB, 크롤링, 클라우드소싱 데이터 수집을 하나의 플랫폼에서 통합 관리합니다.
- 원천 데이터와 처리 이력을 보존해 언제든 재처리할 수 있습니다.
- 사용자가 웹에서 원천 데이터, staging 데이터, master 데이터를 확인할 수 있습니다.
- SQL 기반 정제와 승인 기반 운영 반영을 제공합니다.
- Visual ETL Designer로 사각형 노드 기반 데이터 흐름을 설계합니다.
- 배치, 실시간 처리, 재시도, 오류 격리, 관제 대시보드를 제공합니다.
- 대용량 데이터를 고려한 파티셔닝, 큐, 트랜잭션, 중복 방지 구조를 적용합니다.

3. 전체 아키텍처

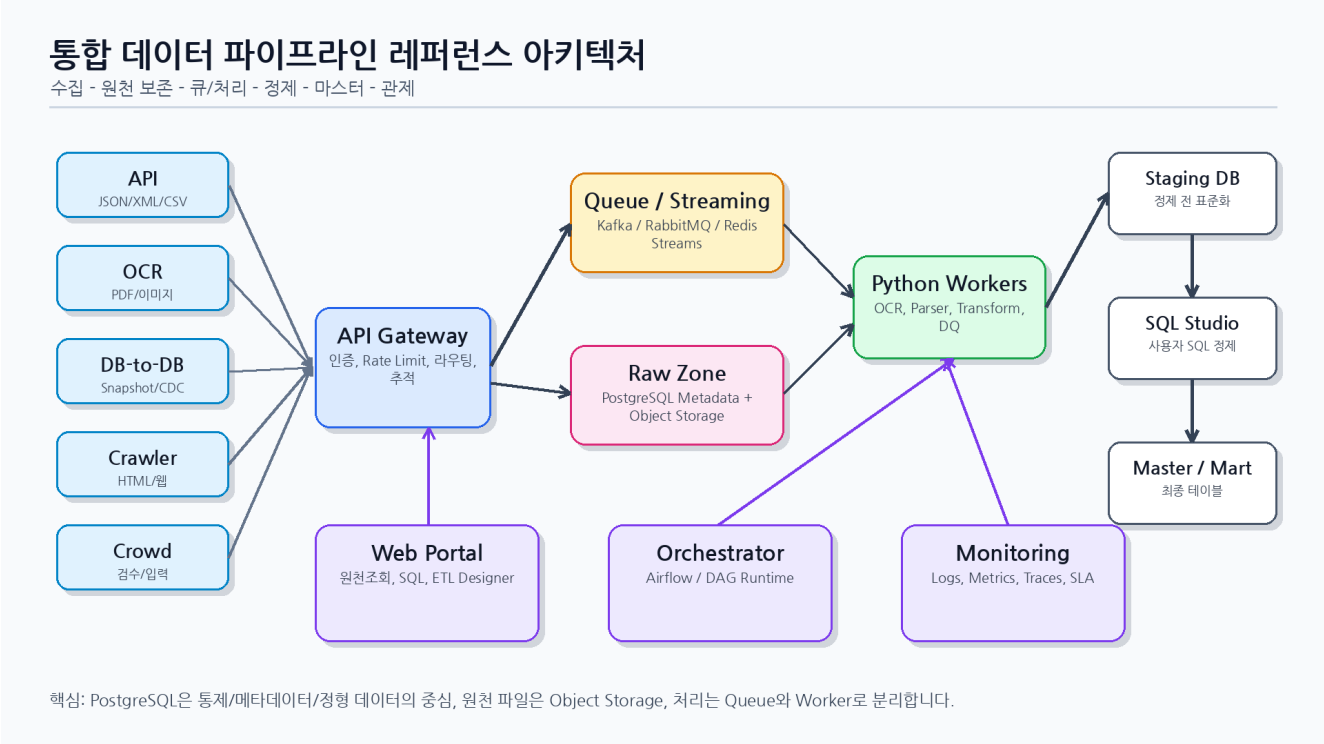

그림 2. 수집 - 원천 보존 - 큐/처리 - 정제 - 마스터 - 관제 구조

3.1 구성 원칙

원칙	설명
수집과 처리 분리	수집기는 원천 저장과 이벤트 발행만 담당하고, 전처리/정제는 Worker가 수행합니다.
원천 보존	원본 파일, 원본 JSON, HTML, OCR 전 이미지, 응답 헤더를 덮어쓰지 않습니다.
재처리 가능성	입력, 출력, 파라미터, 코드 버전, 실행 상태를 모두 기록합니다.
중복 방지	content_hash, idempotency_key, source별 business_key를 사용합니다.
확장성	Queue와 Worker를 통해 수집량 증가 시 수평 확장합니다.
관측 가능성	로그, 메트릭, 트레이스, 데이터 품질 결과, SLA를 통합 관제합니다.

3.2 주요 컴포넌트

영역	권장 기술
Backend API	FastAPI
운영/메타데이터 DB	PostgreSQL
원천 파일 저장	S3, MinIO, 클라우드 Object Storage
Queue/Streaming	Kafka, RabbitMQ, Redis Streams 중 선택
Batch Orchestration	Apache Airflow 또는 자체 DAG Runtime
Worker	Python Worker, Celery, Kubernetes Job
SQL 정제	PostgreSQL SQL, dbt Core, 자체 SQL Studio

영역	권장 기술
Dashboard	Grafana, Superset, 자체 React Dashboard
Visual ETL	React Flow 기반 커스텀 UI
Observability	OpenTelemetry, Prometheus, Grafana, Loki/ELK

4. 데이터 수집 기능 설계

4.1 API 수집

외부 시스템이 JSON, XML, CSV, 파일을 전송하는 표준 수집 경로입니다. API Gateway는 인증, rate limit, schema validation, request tracing, routing을 수행하고 내부 수집 서비스는 원천 저장과 이벤트 발행을 담당합니다.

- API Key, OAuth2, JWT, mTLS 지원
- request body 원본 저장
- JSON Schema 검증
- idempotency key 기반 중복 방지
- 대용량 파일은 pre-signed URL 방식 사용
- 수집 성공 후 큐 이벤트 발행
- 요청/응답/오류 감사 로그 저장

```
POST /v1/ingest/api/{source_code}
POST /v1/ingest/file/{source_code}
POST /v1/ingest/bulk/{source_code}
GET  /v1/ingest/jobs/{job_id}
```

4.2 OCR 수집

PDF, 스캔 이미지, 문서 이미지를 업로드하고 OCR 결과를 구조화합니다. OCR은 원천 이미지와 분리하여 텍스트, 좌표, 신뢰도, 엔진 버전을 저장합니다. confidence가 낮은 항목은 클라우드소싱 검수로 보내는 구조가 적합합니다.

```
파일 업로드
→ 바이러스 검사
→ 파일 해시 계산
→ Object Storage 저장
→ 문서 유형 판별
→ OCR 실행
→ 텍스트/좌표/신뢰도 저장
→ 구조화 추출
→ 품질 검수 또는 staging 적재
```

4.3 DB-to-DB 수집

방식	설명	용도
Snapshot	전체 테이블 복사	초기 적재, 기준 데이터 동기화
Incremental	updated_at, sequence 기준 증분	일반 운영 배치
CDC	DB 변경 로그 기반 수집	실시간 또는 준실시간 동기화

4.4 크롤링 수집

- robots.txt 및 약관 준수
- 도메인별 rate limit
- User-Agent 관리
- HTML 원본 저장
- 페이지 스냅샷 버전 관리
- 실패 URL 재시도
- 파서 버전 관리
- 캡차/로그인 사이트는 합법적 권한 범위에서만 처리

```
crawl_seed
→ crawl_request
→ crawler_worker
→ raw_web_page
→ html_parser
→ extracted_record
→ staging_table
```

4.5 클라우드소싱 수집

OCR 결과 검수, 사람이 판단해야 하는 데이터 입력, 라벨링 작업을 처리합니다. 이중 검수, 작업자 품질 점수, 충돌 해결, 관리자 승인 기능이 필요합니다.

5. 데이터 수명주기

그림 3. 원천 보존과 재처리를 고려한 데이터 수명주기

```
raw
→ staging
→ standard
→ dedup
→ match/merge
→ quality check
→ master
→ mart
```

최종 마스터 테이블만 보는 것이 아니라, 원천, 정제, 품질, 마스터 반영까지 lineage를 추적하는 것이 중요합니다.

6. PostgreSQL DB 스키마 설계

그림 4. PostgreSQL 스키마 계층 설계

6.1 스키마 분리

스키마	역할
ctl	시스템 설정, 소스 정의, 커넥터 정의
raw	원천 데이터 메타데이터, 원본 JSON, 파일 참조
stg	정제 전 임시/표준화 데이터
dq	데이터 품질 검사 결과
wf	시각적 워크플로우 정의
run	실행 이력, 노드 상태, 로그
mart	최종 마스터/분석 테이블
audit	보안, 접근, SQL 실행 감사

6.2 핵심 테이블 DDL

데이터 소스 정의

```
CREATE SCHEMA IF NOT EXISTS ctl;

CREATE TABLE ctl.data_source (
```

```
    source_id          BIGSERIAL PRIMARY KEY,
    source_code        TEXT NOT NULL UNIQUE,
    source_name        TEXT NOT NULL,
    source_type        TEXT NOT NULL CHECK (
        source_type IN ('API', 'OCR', 'DB', 'CRAWLER', 'CROWD')
    ),
    owner_team         TEXT,
    is_active          BOOLEAN NOT NULL DEFAULT TRUE,
    config_json        JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

수집 작업 이력

```
CREATE SCHEMA IF NOT EXISTS run;

CREATE TABLE run.ingest_job (
    job_id             BIGSERIAL PRIMARY KEY,
    source_id          BIGINT NOT NULL REFERENCES ctl.data_source(source_id),
    job_type           TEXT NOT NULL CHECK (
        job_type IN ('ON_DEMAND', 'SCHEDULED', 'RETRY', 'BACKFILL')
    ),
    status             TEXT NOT NULL CHECK (
        status IN ('PENDING', 'RUNNING', 'SUCCESS', 'FAILED', 'CANCELLED')
    ),
    requested_by       TEXT,
    started_at         TIMESTAMPTZ,
    finished_at        TIMESTAMPTZ,
    error_message      TEXT,
    parameters         JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

원천 데이터 테이블

```
CREATE SCHEMA IF NOT EXISTS raw;

CREATE TABLE raw.raw_object (
    raw_object_id      BIGSERIAL,
    source_id          BIGINT NOT NULL REFERENCES ctl.data_source(source_id),
    job_id             BIGINT REFERENCES run.ingest_job(job_id),
    object_type        TEXT NOT NULL CHECK (
        object_type IN ('JSON', 'XML', 'CSV', 'HTML', 'PDF', 'IMAGE', 'DB_ROW')
    ),
    object_uri         TEXT,
    payload_json       JSONB,
    content_hash       TEXT NOT NULL,
    idempotency_key    TEXT,
    received_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    partition_date     DATE NOT NULL DEFAULT CURRENT_DATE,
    status             TEXT NOT NULL DEFAULT 'RECEIVED',
    PRIMARY KEY (raw_object_id, partition_date)
) PARTITION BY RANGE (partition_date);

CREATE TABLE raw.raw_object_2026_04
PARTITION OF raw.raw_object
FOR VALUES FROM ('2026-04-01') TO ('2026-05-01');
```

OCR 결과 테이블

```
CREATE TABLE raw.ocr_result (
    ocr_result_id      BIGSERIAL PRIMARY KEY,
    raw_object_id      BIGINT NOT NULL,
    partition_date     DATE NOT NULL,
    page_no            INTEGER,
    text_content       TEXT,
    confidence_score   NUMERIC(5,2),
    layout_json        JSONB,
    engine_name        TEXT,
    engine_version     TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);
```

Staging 표준 레코드

```
CREATE SCHEMA IF NOT EXISTS stg;

CREATE TABLE stg.standard_record (
    record_id          BIGSERIAL PRIMARY KEY,
    source_id          BIGINT NOT NULL,
    raw_object_id      BIGINT,
    entity_type        TEXT NOT NULL,
    business_key       TEXT,
    record_json        JSONB NOT NULL,
    valid_from         TIMESTAMPTZ,
    valid_to           TIMESTAMPTZ,
    quality_score      NUMERIC(5,2),
    load_batch_id      BIGINT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

마스터 테이블

```
CREATE SCHEMA IF NOT EXISTS mart;

CREATE TABLE mart.master_entity (
    master_id          BIGSERIAL PRIMARY KEY,
    entity_type        TEXT NOT NULL,
    business_key       TEXT NOT NULL,
    canonical_json     JSONB NOT NULL,
    source_count       INTEGER NOT NULL DEFAULT 1,
    confidence_score   NUMERIC(5,2),
    first_seen_at      TIMESTAMPTZ NOT NULL,
    last_seen_at       TIMESTAMPTZ NOT NULL,
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (entity_type, business_key)
);
```

7. 큐, 스택, 이벤트 처리 설계

7.1 권장 Topic

```
ingest.requested
ingest.received
file.uploaded
ocr.requested
ocr.completed
crawler.requested
crawler.completed
dbcdc.changed
staging.ready
transform.requested
transform.completed
dq.failed
dead.letter
```

7.2 처리 원칙

항목	설계
중복 방지	content_hash, idempotency_key, source별 unique key
재시도	exponential backoff
실패 격리	dead-letter queue
순서 보장	같은 business_key는 같은 partition key 사용

항목	설계
대용량 처리	chunk 단위 처리
트랜잭션	DB 저장 성공 후 event outbox 발행
장기 작업	job/node execution 상태 테이블로 관리

7.3 Outbox Pattern

```
1. raw_object insert
2. event_outbox insert
3. transaction commit
4. outbox publisher가 Queue/Kafka로 발행
5. 발행 성공 시 published_at 업데이트
```

```
CREATE TABLE run.event_outbox (
    event_id           BIGSERIAL PRIMARY KEY,
    aggregate_type     TEXT NOT NULL,
    aggregate_id       TEXT NOT NULL,
    event_type         TEXT NOT NULL,
    payload_json       JSONB NOT NULL,
    status             TEXT NOT NULL DEFAULT 'PENDING',
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at       TIMESTAMPTZ
);
```

8. Visual ETL Designer 설계

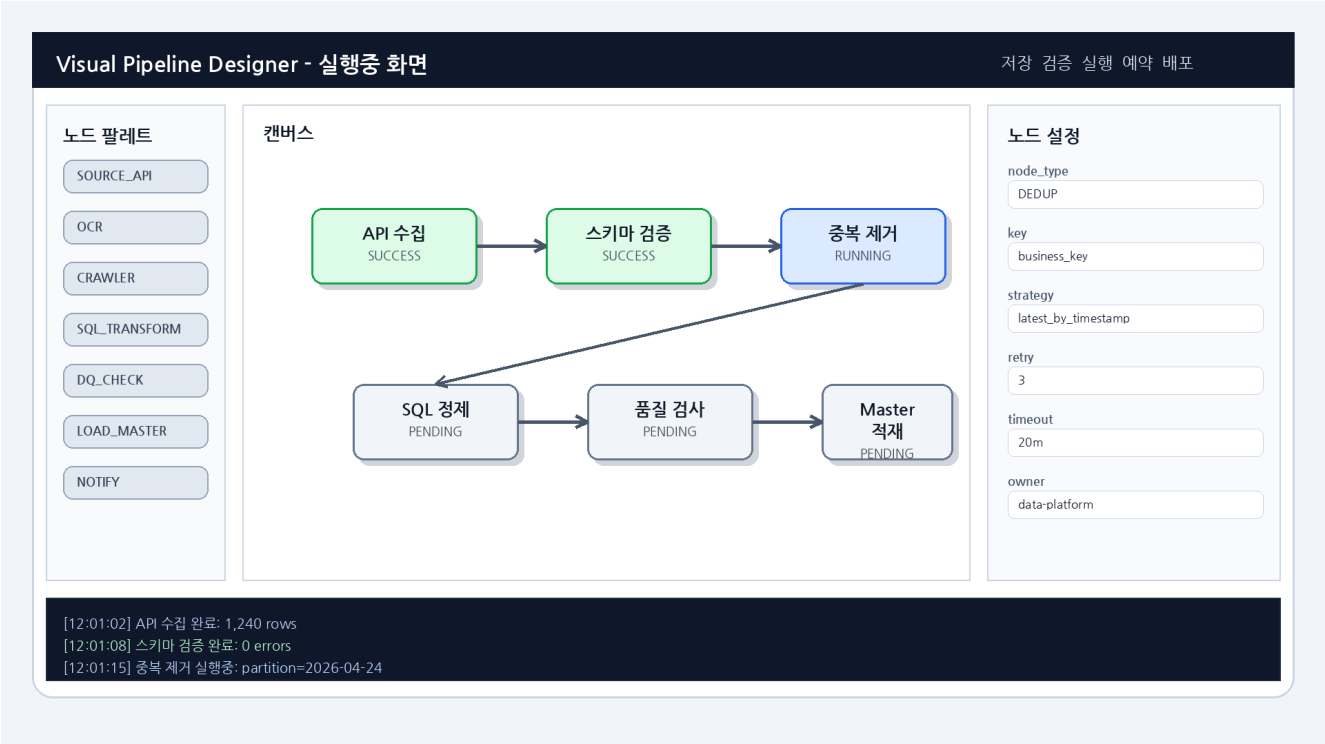

그림 5. SAS Enterprise 스타일의 Visual Pipeline Designer 실행 화면 예시

8.1 사용자 경험

사용자는 왼쪽 팔레트에서 노드를 끌어와 캔버스에 배치하고, 선으로 연결합니다. 노드 설정은 우측 패널에서 정의하고, 실행 로그는 하단 패널에서 실시간으로 확인합니다.

```
[API 수집 노드]
        ↓
[스키마 검증 노드]
```

```
        ↓
[중복 제거 노드]
        ↓
[SQL 정제 노드]
        ↓
[품질 검사 노드]
        ↓
[마스터 적재 노드]
```

상태	의미
READY	실행 대기
RUNNING	실행중
SUCCESS	완료
FAILED	오류
SKIPPED	조건 불충족
RETRYING	재시도중
CANCELLED	취소

8.2 Pipeline 정의 테이블

```
CREATE SCHEMA IF NOT EXISTS wf;

CREATE TABLE wf.pipeline (
    pipeline_id        BIGSERIAL PRIMARY KEY,
    pipeline_name      TEXT NOT NULL,
    description        TEXT,
    version_no         INTEGER NOT NULL DEFAULT 1,
    status             TEXT NOT NULL DEFAULT 'DRAFT',
    created_by         TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE wf.pipeline_node (
    node_id            BIGSERIAL PRIMARY KEY,
    pipeline_id        BIGINT NOT NULL REFERENCES wf.pipeline(pipeline_id),
    node_key           TEXT NOT NULL,
    node_type          TEXT NOT NULL,
    node_name          TEXT NOT NULL,
    position_x         NUMERIC,
    position_y         NUMERIC,
    config_json        JSONB NOT NULL DEFAULT '{}'::jsonb,
    UNIQUE (pipeline_id, node_key)
);

CREATE TABLE wf.pipeline_edge (
    edge_id            BIGSERIAL PRIMARY KEY,
    pipeline_id        BIGINT NOT NULL REFERENCES wf.pipeline(pipeline_id),
    from_node_key      TEXT NOT NULL,
    to_node_key        TEXT NOT NULL,
    condition_expr     TEXT,
    config_json        JSONB NOT NULL DEFAULT '{}'::jsonb
);
```

8.3 실행 상태 테이블

```
CREATE TABLE run.pipeline_run (
    pipeline_run_id    BIGSERIAL PRIMARY KEY,
    pipeline_id        BIGINT NOT NULL REFERENCES wf.pipeline(pipeline_id),
    version_no         INTEGER NOT NULL,
    status             TEXT NOT NULL DEFAULT 'PENDING',
```

```
    triggered_by       TEXT,
    started_at         TIMESTAMPTZ,
    finished_at        TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE run.node_run (
    node_run_id        BIGSERIAL PRIMARY KEY,
    pipeline_run_id    BIGINT NOT NULL REFERENCES run.pipeline_run(pipeline_run_id),
    node_key           TEXT NOT NULL,
    status             TEXT NOT NULL DEFAULT 'PENDING',
    input_count        BIGINT DEFAULT 0,
    output_count       BIGINT DEFAULT 0,
    error_count        BIGINT DEFAULT 0,
    started_at         TIMESTAMPTZ,
    finished_at        TIMESTAMPTZ,
    error_message      TEXT,
    metrics_json       JSONB NOT NULL DEFAULT '{}'::jsonb
);
```

9. SQL 정제 서비스 설계

9.1 주요 기능

- SQL Editor와 미리보기 실행
- 실행 결과 row limit와 timeout
- 실행 계획 확인
- SQL 버전 관리
- 승인 후 운영 반영
- lineage 자동 추적
- 위험 쿼리 차단
- 사용자별 sandbox schema
- read-only role과 transform role 분리

9.2 SQL 보안 정책

```
금지 또는 제한해야 할 작업:

DROP
TRUNCATE
ALTER
권한 없는 schema 접근
운영 mart 테이블 직접 UPDATE
무제한 SELECT
외부 경로로 COPY TO
장시간 lock을 유발하는 쿼리

SQL 작성
→ 정적 분석
→ 권한 확인
→ sandbox 실행
→ 미리보기
→ 승인
→ 운영 배치 등록
```

10. 배치 및 관제 대시보드

10.1 배치 유형

유형	설명
정기 배치	매일/매시간 실행
수동 실행	사용자가 버튼으로 실행
재처리	실패 데이터만 재처리
Backfill	과거 기간 재수집
CDC 실시간	변경 이벤트 기반
SLA 배치	제한 시간 내 완료 여부 추적

10.2 관제 지표

영역	주요 지표
수집	source별 수집 건수, 실패율, 평균 지연
OCR	페이지 수, 평균 신뢰도, 수동 검수율
크롤링	URL 성공률, HTTP status, 차단율
DB-to-DB	CDC lag, snapshot duration
큐	topic lag, consumer lag, retry count
SQL 정제	실행 시간, scan rows, output rows
데이터 품질	null 비율, 중복률, 규칙 실패 수
시스템	CPU, memory, disk, DB connection
사용자	SQL 실행 이력, 승인 이력, 다운로드 이력

11. 대용량 데이터 처리 전략

11.1 PostgreSQL에 저장할 데이터

- 수집 메타데이터
- 정형 staging 데이터
- 마스터 테이블
- 작업 이력
- 품질 검사 결과
- 승인/감사 로그
- 사용자가 조회할 운영성 데이터

11.2 Object Storage에 저장할 데이터

- PDF

- 이미지
- HTML 원본
- 대형 CSV
- 크롤링 스냅샷
- OCR layout JSON 대용량 파일
- 원본 압축 파일

11.3 PostgreSQL 튜닝 전략

- 날짜 기준 partition
- source_id + 날짜 복합 인덱스
- JSONB에는 GIN index
- 시간순 append 테이블에는 BRIN index
- 오래된 raw 데이터는 archive partition으로 이동
- 운영 조회용 summary table 생성
- VACUUM, ANALYZE, autovacuum 튜닝
- connection pooler 사용
- 읽기 replica 분리

12. 데이터 품질 관리

12.1 품질 규칙

유형	예
필수값	고객ID는 NULL 불가
형식	사업자번호 형식
범위	금액은 0 이상
참조	코드값이 기준 코드에 존재
중복	business_key 중복 여부
일관성	시작일 <= 종료일
분포	전일 대비 건수 급감 탐지
OCR 품질	confidence score 임계치

12.2 품질 테이블

```
CREATE SCHEMA IF NOT EXISTS dq;

CREATE TABLE dq.quality_rule (
    rule_id           BIGSERIAL PRIMARY KEY,
    rule_name         TEXT NOT NULL,
    target_schema     TEXT NOT NULL,
    target_table      TEXT NOT NULL,
```

```
    rule_type          TEXT NOT NULL,
    rule_sql           TEXT NOT NULL,
    severity           TEXT NOT NULL CHECK (severity IN ('INFO', 'WARN', 'ERROR')),
    is_active          BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE dq.quality_result (
    result_id          BIGSERIAL PRIMARY KEY,
    rule_id            BIGINT NOT NULL REFERENCES dq.quality_rule(rule_id),
    pipeline_run_id    BIGINT,
    checked_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    status             TEXT NOT NULL CHECK (status IN ('PASS', 'FAIL')),
    failed_count       BIGINT DEFAULT 0,
    sample_json        JSONB
);
```

13. 웹서비스 화면 설계

13.1 주요 메뉴

```
1. 홈 대시보드
2. 데이터 소스 관리
3. 수집 작업 현황
4. 원천 데이터 조회
5. OCR 결과 검수
6. 크롤링 작업 관리
7. 클라우드소싱 작업함
8. SQL 정제 Studio
9. Visual Pipeline Designer
10. 배치 스케줄 관리
11. 품질 검사 결과
12. 마스터 데이터 조회
13. 시스템 관제
14. 감사 로그
15. 사용자/권한 관리
```

13.2 화면 구성

화면	핵심 기능
홈 대시보드	전체 수집량, 실패율, SLA, 최근 오류
소스 관리	API/OCR/DB/크롤러/Crowd 소스 등록
원천 조회	raw_object, 파일 미리보기, payload 확인
OCR 검수	원본 이미지와 OCR 결과 비교
SQL Studio	staging 조회, SQL 작성, 미리보기, 승인 요청
Visual ETL	노드 연결, 실행, 상태 확인
배치 관리	스케줄, 수동 실행, 재실행, Backfill
관제	큐 lag, worker 상태, DB 성능, 오류 로그
감사	접근 이력, SQL 실행 이력, 다운로드 이력

14. 보안 설계

14.1 인증/인가

- SSO/OIDC
- OAuth2/JWT
- Role-Based Access Control
- 데이터 소스별 접근권한
- SQL 실행권한 분리
- 관리자 승인 workflow

14.2 데이터 보안

- 원천 파일 암호화
- DB 컬럼 암호화 또는 마스킹
- 개인정보 탐지
- 다운로드 통제
- 감사 로그
- Row Level Security 검토
- Secret Manager 사용
- API Key rotation

14.3 SQL 실행 보안

```
SQL 작성
→ 정적 분석
→ 권한 확인
→ sandbox 실행
→ 미리보기
→ 승인
→ 운영 배치 등록
```

15. API Gateway 설계

API Gateway는 외부 수집 요청을 내부 서비스에 직접 노출하지 않고 인증, rate limit, quota, request size 제한, request tracing, IP allowlist, schema validation, routing, API versioning, 장애 격리를 담당합니다.

```
/v1/sources
/v1/ingest
/v1/files
/v1/ocr
/v1/crawlers
/v1/pipelines
/v1/pipeline-runs
/v1/sql
/v1/quality
/v1/monitoring
/v1/admin
```

16. MVP 및 구축 로드맵

Phase	구축 항목
Phase 1. Core Foundation	FastAPI 기반 수집 API, PostgreSQL 메타데이터 DB, Object Storage 연동, raw/stg/mart 기본 스키마, 수집 작업 이력, 기본 웹 대시보드, 수동 SQL 정제

Phase	구축 항목
Phase 2. Pipeline Runtime	Queue 도입, Python Worker, 재시도/dead-letter, Airflow 연동 또는 자체 DAG 실행기, 배치 스케줄러, 노드 실행 상태 관리
Phase 3. Visual ETL	React Flow 기반 캔버스, 노드/엣지 저장, SQL 노드, OCR 노드, 마스터 적재 노드, 실행 상태 실시간 표시
Phase 4. Enterprise	CDC, 데이터 품질 엔진, lineage, 권한/승인 workflow, 클라우드소싱 검수, 대용량 partition/archive, 통합 관제, 고가용성/재해복구

17. 구현 우선순위

우선순위	항목	이유
1	raw_object, ingest_job, data_source	원천 보존과 수집 이력의 기반
2	Object Storage 연동	PDF/이미지/대용량 파일을 DB에 직접 넣지 않기 위함
3	Queue + Worker	처리 확장성과 오류 격리
4	SQL Studio	사용자가 직접 정제할 수 있는 핵심 기능
5	pipeline/node_run	Visual ETL 상태 표시의 기반
6	품질 검사	마스터 반영 전 신뢰성 확보
7	관제 대시보드	운영 안정성 확보
8	권한/감사	엔터프라이즈 운영 필수

18. 최종 제안 요약

이 시스템은 단순한 ETL 도구가 아니라 데이터 수집, 원천 보존, 정제, 품질, 마스터 반영, 관제, 재처리, 감사를 한 플랫폼 안에서 연결하는 데이터 운영 체계입니다.

```
Python/FastAPI 수집 API
+ PostgreSQL 메타데이터/정형 데이터 DB
+ Object Storage 원천 파일 저장
+ Queue/Streaming 이벤트 처리
+ Python Worker 정제/전처리
+ Airflow 또는 자체 DAG Runtime
+ React Flow 기반 Visual ETL Designer
+ SQL Studio
+ Grafana/Superset/자체 관제 대시보드
```

가장 중요한 설계 관점: 데이터를 많이 넣는 시스템이 아니라, 어떤 데이터가 언제 어디서 들어와 어떤 규칙으로 정제되어 어떤 마스터 테이블에 반영되었는지 끝까지 추적 가능한 플랫폼을 만드는 것입니다.

19. 참고 기술 문서

- PostgreSQL Documentation: https://www.postgresql.org/docs/
- FastAPI Documentation: https://fastapi.tiangolo.com/
- Apache Airflow Documentation: https://airflow.apache.org/docs/
- Apache Kafka Documentation: https://kafka.apache.org/documentation/

- OpenTelemetry Documentation: https://opentelemetry.io/docs/
- Apache Superset Documentation: https://superset.apache.org/
- React Flow Documentation: https://reactflow.dev/
- MinIO Documentation: https://docs.min.io/